

포인터 보고서

김상연

1. 포인터에 대해 공부한 내용

(1) 포인터 산술의 제한적 활용

포인터에 정수를 더하거나 빼는 건 알고 있었는데, 강의 자료를 보고 명확한 제한이 있다는 걸 알게 됐다.

포인터에 offset(정수)을 더하거나 뺄 수 있다. $p1 + 3$ 은 $p1$ 이후 3번째 요소의 시작주소를 의미한다.

타입이 같은 포인터끼리는 뺄 수 있다. $p2 - p1$ 은 두 포인터 사이의 요소 개수를 의미한다. 단, 두 포인터를 서로 더하는 건 불가능하다.

$*$, $/$, $\%$ 연산자는 포인터를 피연산자로 취할 수 없다.

$p2 - p1$ 은 몇 칸 떨어져 있는가라는 물리적으로 의미 있는 값이 나오지만, $p2 + p1$ 은 두 주소를 더한 결과가 메모리 상 아무 의미도 없는 위치이기 때문에 애초에 언어 차원에서 막혀있다는 걸 이해했다. 포인터 연산은 정수 연산의 축소판이 아니라 메모리 상에서 의미 있는 이동만 허용하도록 설계되어 있다는 게 핵심이었다.

(2) 배열과 함수 - 배열을 넘기는 세 가지 방식

배열을 함수 인자로 넘길 때 call by reference가 쓰인다는 걸 다시 정리했다. 함수 선언부를 다음 두 방식으로 쓸 수 있다.

`void Function(int array[], int n);` (배열 형태)

`void Function(int* array, int n);` (포인터 형태)

호출할 때도 `Function(array, 5)`, `Function(&array, 5)`, `Function(&array[0], 5)` 모두 배열의 시작 주소를 넘긴다는 점에서 동일하게 동작한다. 배열은 함수 인자로 넘어가는 순간 실질적으로 포인터로 취급된다는 게 이 부분의 핵심이었고, 그래서 함수는 배열이 어디서 끝나는지 스스로 알 방법이 없다는 것도 함께 배웠다. 그래서 항상 배열의 길이를 별도 인자로 같이 넘겨야 안전하다는 걸 알게 됐다.

(3) sizeof 연산자와 배열 크기 구하기

`sizeof()`가 피연산자의 크기를 byte 단위로 반환한다는 걸 이용해서 배열 원소 개수를 구하는 공식을 정리했다.

배열 크기 = `sizeof(배열 전체) / sizeof(배열 원소 하나)`

단, 이 공식은 배열이 실제로 선언된 스코프(예: `main` 함수)에서만 정확하다. 배열이 함수 인자로 넘어가서 포인터로 취급되는 곳에서는 통하지 않는다는 것도 함께 확인했다.

(4) 2차원 배열과 함수

2차원 배열을 함수로 넘기는 방법은 `int a[4][3]`, `int a[][3]`, `int (a)[3]`, `int a` 등 여러 형태가 있다는 걸 배웠다. 이 중 `int (*a)[3]`은 3개짜리 `int` 배열을 가리키는 포인터라는 의미이고, `int *a[3]`(포인터 3개로 이루어진 배열)과는 완전히 다른 타입이라는 걸 알게 됐다.

`a`가 3개짜리 배열을 가리키는 포인터이기 때문에 `a+i`는 한 행 전체를 건너뛰고, `*(a+i)`로 그 행의 시작 주소를 얻은 뒤 `+j`로 행 안의 `j`번째 요소로 이동한다. 그래서 `((a+i)+j)`와 `a[i][j]`가

같은 값을 가리킨다는 걸 이해했다. 결국 2차원 배열도 1차원 배열의 배열일 뿐이고, 포인터 연산으로 행과 열을 계산해서 접근하는 것이었다.

(5) 포인터와 문자열

강의 자료에서 다음 두 선언을 비교했다.

```
char s1[] = "Hello World";
```

```
char* s2 = "Hello World";
```

s1은 배열이라서 개별 요소를 자유롭게 수정할 수 있지만(s1[0] = 'h');, 처음 초기화된 크기 이상으로 확장할 수 없다.

s2는 포인터이고, 문자열 리터럴이 읽기 전용 메모리 영역에 저장되기 때문에 개별 요소 수정이 불가능하다. 대신 s2 = "Hello World!!!";처럼 아예 다른 문자열을 가리키게 재할당하는 건 가능하다.

배열은 내용은 바꿀 수 있지만 크기는 못 바꾸고, 포인터는 내용은 못 바꾸지만 다른 곳을 가리킬 수 있다는 대조로 정리했다.

또한 C에서 문자열은 배열 크기와 무관하게 null 종료 문자(\0)를 만날 때까지를 문자열의 끝으로 인식한다는 것. %s는 이 \0을 만날 때까지 자동 출력하지만 %c는 문자 하나만 출력한다는 차이도 배웠다. 문자열 입력의 경우 scanf("%s", s);는 공백을 만나면 입력이 끊기고, 공백을 포함한 문장 전체를 입력받으려면 scanf("%[^\n]", s);를 써야 한다는 것도 정리했다.

(6) 이중 포인터

이중 포인터(char** str)는 함수 안에서 포인터 변수 자체가 가리키는 곳을 바꾸고 싶을 때 사용한다. 일반 포인터를 함수에 넘기면 그 포인터가 가리키는 값은 바꿀 수 있지만, 포인터 변수 자체는 함수 인자로 값 복사되어 전달되기 때문에 함수 밖에서 바뀌지 않는다. 포인터 변수의 주소(&str)를 넘겨야 함수 안에서 그 포인터가 가리키는 곳 자체를 재할당할 수 있다는 걸 배웠다.

2. 과제 시 문제점과 해결방법

(1) 포인터 두 개를 더하려고 함

문제점 : p2 + p1처럼 포인터끼리 더하는 연산을 시도했다가 컴파일 에러가 발생했다.

처음엔 왜 안 되는지 이해가 안 됐다.

```
int arr[5] = {1,2,3,4,5};
```

```
int *p1 = arr;
```

```
int *p2 = arr + 3;
```

```
printf("%d\n", p2 + p1); (컴파일 에러)
```

해결방법 : p2 - p1(뺄셈)은 두 주소 사이에 몇 칸이 있는가라는 의미가 성립하지만, p2 + p1(덧셈)은 결과값이 메모리 상 아무 의미 없는 위치가 되어버린다는 걸 이해하고, 포인터 연산은 뺄셈만 허용된다는 규칙을 확인했다.

(2) 함수 매개변수에서 배열과 포인터를 혼동함

문제점 : array[]로 선언한 매개변수가 배열처럼 동작할 거라 생각했는데, 함수 안에서

sizeof(array)를 찍어보니 배열 전체 크기가 아니라 포인터 크기(8바이트)가 나왔다.

```
void printSize(int array[]) {  
    printf("%lu\n", sizeof(array)); (8, 포인터 크기)
```

} 해결방법 : 배열이 함수 인자로 넘어가는 순간 실제로는 포인터로 변환되어 처리된다는 걸 이해했다. 이후로는 함수 안에서 배열 크기가 필요할 때 sizeof를 쓰지 않고, 크기를 별도 매개변수로 받아서 사용하도록 코드를 고쳤다.

(3) 배열 크기를 넘기지 않아 범위를 벗어난 접근이 발생함

문제점 : 함수에 배열 길이를 넘기지 않고, 배열 크기(5)를 훨씬 초과하는 인덱스에 접근하는 코드를 실행했다.

```
void function1(int list[]) {
```

```
list[100] = 1; (배열 크기 5 초과)
```

} 컴파일은 문제없이 통과됐지만, 실행하니 "Stack around the variable 'list' was corrupted"라는 런타임 오류가 발생했다.

해결방법 : 함수는 배열의 시작 주소만 받을 뿐 배열이 어디서 끝나는지 알 방법이 없다는 걸 깨닫고, 배열 길이를 별도 인자로 함께 전달하도록 수정했다.

```
void function1(int* list, int size) {
```

```
for (int i = 0; i < size; i++)
```

```
printf("%d ", list[i]);
```

3.LLM을 활용하며 배운 것

(1) 개념을 물어볼 때 예시 코드까지 같이 요청하면 이해가 빨랐다

처음엔 그냥 "포인터가 뭐야?" 식으로 물어봤는데 설명이 너무 추상적으로 느껴졌다. 그래서 "코드 예시랑 같이 설명해줘"라고 구체적으로 요청하니 개념과 실제 동작이 바로 연결돼서 이해가 훨씬 빨랐다. 특히 배열과 포인터, 배열 포인터와 포인터 배열처럼 헷갈리는 것들은 비교 코드를 나란히 보여달라고 하니 차이가 명확하게 보였다.

(2) 내가 직접 틀린 코드를 짜보고 그 이유를 물어보는 방식이 훨씬 기억에 남았다

LLM이 처음부터 정답 코드를 주는 것보다, 내가 예상과 다르게 동작하는 코드를 먼저 짜보고 "왜 이렇게 나와?"라고 물어봤을 때가 훨씬 이해가 오래갔다. 예를 들어 함수 안에서 sizeof(array)가 다르게 나오는 것도, 직접 실험해보고 결과를 캡처해서 물어보니 원인을 훨씬 명확하게 설명해줬다. 앞으로도 답을 먼저 구하기보다 직접 실험하고 안 되는 이유를 물어보는 방식으로 공부하는 게 더 효율적일 것 같다.

(3) 강의 자료(PDF)를 같이 첨부하고 질문하니 훨씬 정확한 답을 받을 수 있었다

그냥 텍스트로만 질문했을 때보다, 실제 수업에서 쓴 PDF 자료를 첨부해서 "이 내용 기준으로 설명해줘"라고 하니 강의에서 배운 표현이나 예제와 일치하는 설명을 받을 수 있었다. 특히 배열 포인터 표기(int (*a)[3]) 같은 건 내 기억만으로 설명하면 헷갈렸을 텐데, 자료를 같이 주니까 정확하게 짚어줘서 도움이 됐다.

(4) LLM이 만들어준 설명을 그대로 베끼지 않고 내 언어로 다시 정리하는 과정이 필요했다

처음 LLM이 정리해준 내용을 그대로 제출하려니 표현이 너무 매끄럽고 획일적이어서 오히려 내가 직접 공부한 티가 안 났다. 그래서 내용을 다시 읽고, 내가 실제로 헛갈렸던 부분과 겪은 시행착오를 기준으로 직접 다시 쓰는 과정을 거쳤다. 이 과정에서 오히려 한 번 더 개념을 되짚어보게 되어서, 단순히 정리된 답을 받는 것보다 더 오래 기억에 남는 학습이 됐다.